

DUNGEON CRAWLER: EL DESCENSO AL NÚCLEO

Informe Técnico del Proyecto de Software en Lenguaje ANSI C

Asignatura:	Fundamentos de Programación (TAO207)
Proyecto:	Dungeon Crawler en consola (Valor: 20%)
Integrantes:	Gabriel González Co-Autor del Proyecto
Cédulas de Identidad:	C.I. V-30.123.456 / C.I. V-30.654.321
Sección:	Sección 01
Profesor:	Cátedra de Fundamentos de Programación
Fecha de Entrega:	Septiembre, 2026

RESUMEN EJECUTIVO

El presente informe documenta el diseño, la arquitectura modular y la implementación en ANSI C del sistema *Dungeon Crawler: El Descenso al Núcleo*. El software modela un videojuego de rol y exploración de mazmorras por turnos accionado mediante consola. Integra algoritmos procedimentales controlados por semilla, persistencia de datos mediante archivos de texto (configuración, bestiario, catálogo de objetos, partidas guardadas y registro histórico de puntuaciones), así como mecanismos de combate, gestión de inventario y progresión de personaje.

1. Descripción del Problema

Contexto General y Rol del Desarrollador: En el ámbito de la ingeniería de software y la programación estructurada, el proyecto *El Descenso al Núcleo* asigna al programador el rol de modelar las entidades, la gestión de memoria y el flujo de ejecución de un simulador de exploración de mazmorras subterráneas (Dungeon Crawler) interactivo en consola ANSI C.

Objetivos del Software: El objetivo primordial es ofrecer una experiencia de juego por turnos coherente y desafiante, donde el usuario navega a través de múltiples niveles o pisos, interactúa con entidades enemigas mediante combate táctico, administra recursos de supervivencia en su mochila y busca alcanzar el último cuarto del último piso para lograr la victoria absoluta.

Reglas y Mecánicas del Juego:

- **Navegación Abstracta y Lineal:** La mazmorra se divide en pisos ordenados jerárquicamente, y cada piso en cuartos consecutivos. El avance es estrictamente unidireccional (no se permite retroceder a cuartos anteriores).
- **Estructura de Cuartos:** Todos los pisos comparten una cantidad base de cuartos definida dinámicamente desde un archivo de configuración (`config.txt`). Como regla del sistema, el primer piso posee un cuarto menos que la cantidad base, mientras que el piso final posee un cuarto adicional que alberga el encuentro contra el jefe final.
- **Encuentros y Enemigos:** Cada cuarto contiene un enemigo. En los cuartos ordinarios habitan enemigos de tipo *Comun*, mientras que en el último cuarto de cada piso habita obligatoriamente un enemigo de tipo *Raro* (Jefe).
- **Condición de Victoria:** Derrotar al jefe final ubicado en el último cuarto del último piso.
- **Condición de Derrota:** Que los puntos de vida (HP) del jugador se reduzcan a cero o menos durante cualquier combate.
- **Muerte Permanente (Permadeath):** Al morir o al ganar, la partida actual finaliza de forma definitiva y se elimina el archivo de partida guardada para impedir la restauración indebida del estado.

2. Análisis de la Solución

La arquitectura de la solución se concibió bajo los principios fundamentales de la **programación estructurada y modular** en ANSI C, dividiendo la responsabilidad del sistema en 5 módulos interconectados mediante subprogramas con firma explícita y pasaje de parámetros estricto.

Organización Modular del Sistema:

Módulo	Funcionalidad y Responsabilidad Principal
1. Inicialización e Input	Lectura e inyección de parámetros desde `config.txt`, parseo del catálogo de enemigos desde `bestiario.txt` e inyección de ítems desde `objetos.txt`.
2. Exploración y Navegación	Gestión del mapa lineal de cuartos/pisos, cálculo dinámico del tamaño de piso y selección pseudoaleatoria de enemigos según tipo de cuarto.
3. Combate y Progresión	Motor de combate por turnos, cálculo de fórmula de daño, probabilidad de huida, asignación de botín y verificación de subida de nivel (XP).
4. Gestión de Recursos	Administración del inventario/mochila del jugador y aplicación del tipo de efecto al consumir consumibles.
5. Persistencia y Output	Persistencia de partidas (`partida_guardada.txt`), borrado por muerte permanente y registro acumulativo en `highscore.txt`.

Justificación de Decisiones de Diseño:

- **Aleatoriedad Controlada por Semilla (`srand` / `rand`):** Para garantizar la reproducibilidad y auditoría de las partidas en las pruebas académicas, el generador de números pseudoaleatorios se inicializa con la semilla numérica provista en el archivo `config.txt`.
- **Paso de Parámetros por Referencia (Punteros):** Las variables que sufren modificaciones continuas (como la estructura `EstadoJuego`) se comunican mediante punteros (`EstadoJuego \*estado`), evitando la sobrecarga de copia en la pila de llamadas y garantizando mutabilidad segura.
- **Limpieza de Buffer de Entrada (`limpiarBuffer`):** La mezcla del uso de `scanf` con lectura de cadenas provoca caracteres residuales en el buffer del teclado (`stdin`). Se implementó una rutina auxiliar que consume caracteres hasta encontrar `\n` o `EOF`.
- **Manejo Seguro de Archivos:** Todas las aperturas mediante `fopen` cuentan con validación de puntero nulo (`NULL`), emitiendo mensajes de error informativos y deteniendo la ejecución antes de incurrir en fallos de segmentación.

3. Estructuras de Datos

El núcleo del software reposa sobre tres registros (`struct`) fundamentales definidos en lenguaje ANSI C, complementados con arreglos estáticos para la manipulación en memoria de catálogos y colecciones.

1. **Registro de Objeto (`struct Objeto`):** Almacena la definición de los ítems del catálogo y los elementos guardados en la mochila.
-

```
typedef struct {
    int id;
    char nombre[50];
    int valor;           // Puntos de curación o incremento de atributo
    char tipoEfecto[20]; // "Cura_Vida", "Buff_Dano", "Buff_Defensa", "Cura_Max"
} Objeto;
```

2. Registro de Entidad Enemiga (`struct Entidad`): Representa a los adversarios cargados desde el bestiario.

```
typedef struct {
    int id;
    char nombre[50];
    int vida;
    int dano;
    int defensa;
    char tipo[20];      // "Comun" o "Raro"
} Entidad;
```

3. Registro de Estado del Juego (`struct EstadoJuego`): Mantiene el estado dinámico completo del jugador durante la aventura.

```
typedef struct {
    int hp;
    int hpMax;
    int ataque;
    int defensa;
    int nivel;
    int xp;
    int piso;
    int cuarto;
    Objeto inventario[100]; // Arreglo estático de mochila
    int cantObjetos;
    int capacidadInventario;
    bool enJuego;
    bool victoria;
} EstadoJuego;
```

4. Algoritmos Implementados

A continuación se presentan los subprogramas principales del sistema expresados mediante texto estructurado y lenguaje pseudoformal.

Subprograma 1: Carga de Configuración e Inicialización (`cargarConfiguracion`)

ENTRADA: Ruta del archivo config.txt, Puntero estado (EstadoJuego*)

SALIÓ: Estado del juego parametrizado en memoria

ALGORITMO:

1. Abrir archivo config.txt en modo lectura ("r").
2. Leer semilla, totalPisos, cuartosBase.
3. Iniciar randomizador: llamar a srand(semilla).
4. Leer hpInicial, ataqueInicial, defensaInicial, capacidadInventario.
5. Asignar atributos a estado: hp = hpInicial, hpMax = hpInicial, nivel = 1, xp = 0, piso = 1, cuarto = 0.
6. Cerrar archivo config.txt.

Subprograma 2: Selección de Enemigos (`seleccionarEnemigo`)

ENTRADA: Arreglo bestiario[], totalEnemigos (int), esCuartoFinal (bool)

SALIDA: Entidad seleccionada

ALGORITMO:

1. Crear arreglo filtrado[] de capacidad totalEnemigos.
2. Inicializar contadorFiltrados = 0.
3. Para cada i desde 0 hasta totalEnemigos - 1:
 - a. Si esCuartoFinal ES VERDADERO y bestiario[i].tipo == "Raro":
Agregar bestiario[i] a filtrado[] y sumar 1 a contadorFiltrados.
 - b. Si esCuartoFinal ES FALSO y bestiario[i].tipo == "Comun":
Agregar bestiario[i] a filtrado[] y sumar 1 a contadorFiltrados.
4. Generar índiceAleatorio = rand() % contadorFiltrados.
5. Retornar filtrado[índiceAleatorio].

Subprograma 3: Motor de Combate por Turnos y Fórmulas (`iniciarCombate`)

Durante el combate por turnos, se aplican estrictamente las siguientes fórmulas matemáticas:

- **Fórmula de Daño:** $\text{DanoReal} = \max(1, \text{AtaqueAtacante} - \text{DefensaDefensor})$. Garantiza que todo golpe cause como mínimo 1 punto de daño.
- **Probabilidad de Huida:** $\text{ProbHuida} = 50 + (\text{DefensaJugador} - \text{DanoEnemigo}) * 2$. El resultado se acota mediante un operador de sujeción (clamp) en el rango $[10\%, 90\%]$.
- **Progresión de Nivel:** Umbral de experiencia requerido: $\text{XP_Requerida} = \text{NivelActual} * 100$. Al subir de nivel, se cura totalmente la salud del jugador ($\text{hp} = \text{hpMax} + 20$), incrementando hpMax en +20 y mejorando Ataque y Defensa base en +5 puntos cada uno.

5. Pruebas Realizadas

Para garantizar la calidad de software y el cumplimiento cabal de las especificaciones, se ejecutó una batería de casos de prueba exhaustivos:

Caso de Prueba	Entrada / Condición	Comportamiento Observado y Resultado
1. Inicialización e Inputs	Semilla 42 en `config.txt`, lectura de `bestiario.txt` y `objetos.txt`.	EXITOSO: Carga correcta de variables, asignación exacta de semilla con `srand` y catálogo en memoria.

2. Generación de Jefes	Avance al cuarto final de cualquier piso.	EXITOSO: El algoritmo seleccionó exclusivamente entidades con atributo `tipo = Raro`.
3. Cálculo de Daño y Huida	Defensa alta contra enemigo con ataque bajo.	EXITOSO: Se aplicó el daño mínimo (1 HP) y la probabilidad de huida respetó los límites [10%, 90%].
4. Subida de Nivel	Acumular $XP \geq Nivel * 100$.	EXITOSO: Incremento de nivel, recuperación total de vida, +20 HP Máx, +5 Ataque y +5 Defensa.
5. Persistencia y Carga	Guardar en cuarto 2, piso 2. Salir y reanudar.	EXITOSO: Restauración exacta de inventario, estadísticas, piso y cuarto exactos.
6. Muerte Permanente	HP reducido a 0 en combate o finalizar juego.	EXITOSO: Eliminación automática de `partida_guardada.txt` e inyección de datos en `highscore.txt`.

6. Conclusiones y Recomendaciones

Reflexión sobre el Desarrollo: La realización del proyecto *Dungeon Crawler: El Descenso al Núcleo* permitió consolidar de forma práctica los pilares de la programación estructurada en ANSI C estricto. La abstracción de problemas complejos en módulos funcionales independientes demostró ser una práctica indispensable para mantener la mantenibilidad, legibilidad y solidez del código fuente.

Dificultades Superadas: Entre los mayores retos técnicos destacó la gestión segura de buffers de teclado al combinar entradas numéricas y de texto, resuelto exitosamente mediante la rutina `limpiarBuffer()`. Asimismo, la manipulación explícita de archivos para garantizar la persistencia de estados y la implementación rigurosa de la regla de permadeath exigieron un control meticuloso de los punteros de archivo (`FILE\*`).

Posibles Mejoras Futuras:

- **Interfaz de Usuario Avanzada:** Implementar bibliotecas como `ncurses` o manipulación de secuencias de escape ANSI para ofrecer una interfaz visual en consola con mapa bidimensional activo.
- **Generación Procedural de Mazmorras:** Reemplazar la progresión lineal por algoritmos de generación de laberintos (ej. Árboles de Expansión Mínima o Llenado por Inundación) para mayor rejugabilidad.
- **Sistema Extendido de Clases y Habilidades:** Incorporar árboles de habilidades especiales, maná y estados alterados (veneno, aturdimiento) en las entidades del bestiario.